

eSal — NFC Pairing: The Two Pieces, Explained

A plain-language walkthrough of the tag-writing script and the pairing-wait poller, and why they're two separate things.

The one idea to hold onto through all of this: these two pieces of code run at **completely different times**, for different reasons. Piece 1 runs **once, ever**, by hand, when you're setting up a device. Piece 2 runs **every time the Pi boots**, waiting for something to happen. Keeping that distinction in your head makes everything else in this document make sense.

Table of Contents

1. The big picture — why two pieces at all
2. Piece 1 — the tag-writing script (runs once)
3. Piece 2 — the pairing-wait poller (runs at boot)
4. What happens when a customer taps — and why the Pi does nothing
5. If something doesn't work

1. The big picture

Before either piece of code makes sense, it helps to see the whole sequence of events an NFC tag goes through in this system:

- 1 Device setup (happens once, by you, with a screwdriver in hand):** the Pi already has a `pairingId` (a random ID it generated for itself the first time it ever booted, saved in `pairing_id.txt`). You run Piece 1 — the tag-writing script — one time, which writes a link containing that `pairingId` onto the physical NFC tag.
- 2 Every time the Pi boots after that** (until it's paired), it runs Piece 2 — a loop that keeps asking the backend "has anyone paired with me yet?" This is necessary

because, unlike the QR-code flow, the Pi itself never makes the pairing request — the *shop owner's phone* does, after they tap the tag. The Pi has to find out some other way.

3 The shop owner taps their phone to the tag. Their phone reads the link, opens the eSal app, the app attaches their login, and the app calls the backend directly to complete pairing.

4 The Pi's Piece 2 loop notices pairing succeeded (within a few seconds), grabs its new API key from the backend, and saves it. The Pi is now a fully working device.

5 Later, any customer taps the exact same tag to claim a receipt. The Pi is not involved in this step at all — covered in section 4.

So: one script writes the tag (once), one function waits to hear about pairing (every boot, until paired), and after that, the tag just sits there being read by customers with zero further Pi involvement.

2. Piece 1 — the tag-writing script

Runs once, by hand, during setup

An NFC tag doesn't just "contain a link" the way you might paste a URL into a text file. Phones only recognize a link on a tag if it's wrapped in a specific, standardized format called **NDEF** (NFC Data Exchange Format). This script's whole job is building that exact byte structure and writing it onto the tag through the PN532 reader/writer.

Why the bytes have to look a very particular way

Think of it like an envelope inside an envelope:

Layer	What it is	Why it exists
Outer: the TLV wrapper	A tiny header (<code>0x03</code> , then a length byte) around the whole	This is how any NFC Forum "Type 2" tag (which includes your NTAG215)

	message, ending in a "terminator" byte (<code>0xFE</code>)	marks where its actual data starts and ends in memory
Inner: the NDEF record	A header byte, then "this is type U (URI)", then the actual link	This is how a phone knows <i>what kind</i> of data it just read — a URI, versus a phone number, versus plain text, etc.
Innermost: the URI payload	A single code byte meaning "this starts with https://", followed by the rest of the URL	Saves space — instead of spelling out "https://" every time, there's a 1-byte shorthand for common prefixes

The code, piece by piece

Building the innermost part — the URI record:

```
def build_ndef_uri_message(url: str) -> bytes:
    prefixes = {
        "http://www.": 0x01,
        "https://www.": 0x02,
        "http://": 0x03,
        "https://": 0x04,
    }
    code = 0x00
    rest = url
    for prefix, abbrev_code in prefixes.items():
        if url.startswith(prefix):
            code = abbrev_code
            rest = url[len(prefix):]
            break

    payload = bytes([code]) + rest.encode("utf-8")
    record_type = b"U"

    header = 0xD1
    return bytes([header, len(record_type), len(payload)]) + record_type + payload
```

Walking through this: our URL starts with `"https://"`, so `code` becomes `0x04` and `rest` becomes everything after those 8 characters (e.g. `esal.app/nfc?pairingId=...`). The `payload` is just that one code byte glued to the rest of the URL's actual bytes.

`header = 0xD1` is a fixed value that means, in order: "I am the first record in this message, I am also the last record (there's only one), this is a short record, and my type is a well-known NFC Forum type." You don't need to memorize that byte — it's always the same for a single, simple URI record like this one.

Why bother with the abbreviation code at all? Because tags have limited memory. Your NTAG215 has 504 usable bytes — not a lot. Shaving 7 characters off every single write by using a 1-byte code instead of spelling out "https://" is a real, standard practice, not a micro-optimization nobody uses.

Wrapping it in the TLV envelope:

```
def build_type2_tlv(ndef_message: bytes) -> bytes:
    tlv = bytes([0x03, len(ndef_message)]) + ndef_message
    tlv += bytes([0xFE])
    return tlv
```

`0x03` is a fixed value meaning "an NDEF message follows." The next byte says how many bytes that message is. Then the message itself. Then `0xFE` says "nothing more to read after this." This whole function is just gluing 2 bytes on the front and 1 byte on the end — the actual content was already built by the previous function.

Actually writing it to the physical tag:

```
def write_tag(pn532, data: bytes, start_page: int = 4):
    remainder = len(data) % 4
    if remainder:
        data = data + bytes(4 - remainder)

    page_count = len(data) // 4
    for i in range(page_count):
        page = start_page + i
        block = data[i * 4:(i + 1) * 4]
        if not pn532.ntag2xx_write_block(page, block):
            raise RuntimeError(f"Failed writing page {page}")
    print(f"[+] Wrote {page_count} pages starting at page {start_page}")
```

Here's the one physical quirk this deals with: NTAG215 memory isn't one continuous strip you can write freely — it's organized into fixed **4-byte pages**, and you can only write a whole page at a time. Our TLV-wrapped data is probably not an exact multiple of 4 bytes long, so the first few lines pad it out with zero bytes until it is. Then the loop writes it 4 bytes at a time, starting at **page 4** — pages 0 through 3 are reserved by the tag itself for its UID and configuration, so user data always starts at page 4 on this tag family.

Tying it together — the `main()` function:

```
def main():
    with open(PAIRING_ID_FILE, "r") as f:
        pairing_id = f.read().strip()

    url = f"{NFC_LINK_PREFIX}{pairing_id}"
    tlv_data = build_type2_tlv(build_ndef_uri_message(url))

    i2c = busio.I2C(board.SCL, board.SDA)
    reset_pin = DigitalInOut(RESET_PIN)
    req_pin = DigitalInOut(REQ_PIN)
    pn532 = PN532_I2C(i2c, debug=False, reset=reset_pin, req=req_pin)

    pn532.SAM_configuration()

    uid = None
    while uid is None:
        uid = pn532.read_passive_target(timeout=0.5)

    write_tag(pn532, tlv_data)
```

This reads the Pi's own `pairing_id.txt` (the same ID it already made for itself), builds the full link, turns it into the TLV bytes using the two functions above, connects to the PN532 exactly like your earlier tag-identifying script did, waits until a tag is actually placed near the antenna, and then writes. Nothing here is new machinery — it's the same PN532 connection code you've already used, plus the byte-building functions explained above.

3. Piece 2 — the pairing-wait poller

Runs every boot, until the device is paired

Recall from the QR-code flow: when a customer scans a QR code, the **Pi itself** sends the pairing request to the backend, and gets the API key back directly in that same response — simple, synchronous, done.

NFC pairing doesn't work that way. The **phone** is the one that reads the tag and calls the backend — the Pi is never part of that phone call at all. So the Pi has no direct way to know the moment pairing happens. The only option left is to keep asking: *"hey backend, has anyone paired with my pairingId yet?"* — over and over, a few seconds apart, until the answer is yes.

```
def wait_for_nfc_pairing(pairing_id, poll_interval=3, timeout=None):
    elapsed = 0
    while timeout is None or elapsed < timeout:
        try:
            response = requests.get(
                f"{BACKEND_URL}/devices/pair/status",
                params={"pairingId": pairing_id},
                timeout=10,
            )
            response.raise_for_status()
            data = response.json()
            if data.get("paired") and "apiKey" in data:
                return data["apiKey"]
        except requests.exceptions.RequestException as e:
            print(f"[!] Status check failed, retrying: {e}")
        time.sleep(poll_interval)
        elapsed += poll_interval
    return None
```

Reading this loop in plain language: "ask the backend about this pairingId. If it says paired and hands back an API key, we're done — return it immediately. If the request fails for any reason (network hiccup, backend briefly down), don't crash — just print a warning and try again in a few seconds. Keep doing this forever, unless a timeout was given and we've hit it."

Why check for both "paired" and "apiKey" together? Because the backend's status endpoint only hands back the API key *the first time* it's asked after pairing succeeds — every question after that just says "yes, paired" with no key attached (that's a deliberate security measure, so the key can't be stolen by someone else who happens to read the same tag later). Checking for both means this function only returns once it actually has something useful to save — a bare "yes" with no key isn't treated as success here.

```
def run_nfc_pairing_flow():
    pairing_id = get_or_create_pairing_id()
    red_led.blink(on_time=0.5, off_time=0.5)

    api_key = wait_for_nfc_pairing(pairing_id)

    red_led.off()
    if not api_key:
        red_led.blink(on_time=0.1, off_time=0.1)
        return False

    with open(API_KEY_FILE, "w") as f:
        f.write(api_key)

    green_led.on()
    return True
```

This is the outer wrapper, and it should look familiar — it's deliberately built to feel almost identical to the existing QR pairing flow: slow red blink while waiting, LED off once something happens, fast red blink on failure, solid green plus saving the key on success. The *only* thing that's different from the QR version is what happens in the middle — instead of listening on the UART for a scanned code, it calls the polling function above and waits for that to return something.

4. What happens when a customer taps — and why the Pi does nothing

This is worth stating plainly because it's easy to assume there must be more Pi-side code somewhere for this: **there isn't**. Once Piece 1 has written the tag one time, the Pi's job with respect to that tag is finished, permanently. Every time a customer taps it afterward:

Their phone reads the same link off the tag → opens the eSal app → the app attaches *that customer's* login → the app calls the backend directly, asking it to find the most recent unclaimed receipt for that device and assign it to them.

The Pi is never contacted, never polled, never involved. It doesn't need to know a receipt was claimed, the same way it never needed to know that for the QR-based claiming flow's edge cases either — the backend and the database are what track claim status; the Pi only ever cares about "what's the most recent receipt I created," which gets naturally overwritten by the next one regardless of what happened to the last one.

5. If something doesn't work

Symptom	Likely cause
Write script fails / hangs	Same as your earlier tag-identifying script — check wiring (SDA/SCL/RSTPD_N/REQ) and that the tag is actually within range
Write succeeds, but no generic NFC app recognizes it as a link	The tag's "Capability Container" (page 3) might not be set — consumer NTAG215 cards almost always ship pre-formatted for this, so it's an unlikely but possible first thing to check
Poller runs forever, never succeeds	Confirm the shop owner's phone actually completed the tap-to-pair flow, and that <code>NFC_LINK_PREFIX</code> in the write script matches your app's real Universal Link domain — a placeholder domain with no real app association will just open a browser instead of your app